

Tienda de Códigos Digitales de Videojuegos

Profesor:

Dr. Luis Veas

Estudiantes:

Álvaro Burgos, Patricio Cuevas, Luciano Espinoza,
Felipe Henriquez, Daniela Huenumán

Abstract

El presente trabajo aborda el diseño de una plataforma distribuida para la venta de códigos digitales de videojuegos, orientada a soportar altos niveles de concurrencia y garantizar la consistencia de los datos. El problema principal identificado corresponde a la posibilidad de ventas duplicadas producto de condiciones de carrera durante eventos de alta demanda. Para mitigar este riesgo, se propone una arquitectura basada en microservicios y orientada a eventos (EDA), apoyada en herramientas como Nginx como reverse proxy, RabbitMQ como sistema de mensajería, Redis para gestión de sesiones y caché, PostgreSQL como base de datos relacional y Apache Solr como motor de búsqueda.

La solución incorpora técnicas como particionamiento de datos por región (sharding), control de concurrencia y procesamiento asíncrono de eventos, permitiendo escalar horizontalmente y mejorar la disponibilidad del sistema. Asimismo, se implementa un modelo híbrido de almacenamiento que combina bases de datos relacionales y NoSQL para optimizar el rendimiento en operaciones críticas como autenticación, búsqueda y gestión de carritos de compra.

Finalmente, se definen requerimientos clave del sistema, tales como tiempos de respuesta acotados, consistencia fuerte en transacciones y mecanismos de tolerancia a fallos, con el objetivo de construir una plataforma robusta, segura y escalable que responda a las exigencias del mercado actual de distribución digital.

Index Terms

E-Commerce, Secure Transactions, Distributed Systems, Event-Driven Architecture, Message Queues.

I. INTRODUCCIÓN

El mercado de distribución digital de videojuegos ha experimentado un crecimiento acelerado en los últimos años, impulsado por la alta demanda de acceso inmediato a contenidos y la globalización del consumo digital. Plataformas como Steam, Eneba o G2A han consolidado un modelo basado en la venta de códigos digitales, permitiendo a los usuarios adquirir productos de forma rápida, remota y muchas veces a menor costo.

Sin embargo, este modelo presenta desafíos técnicos significativos, especialmente en contextos de alta demanda. Durante eventos masivos como promociones o fechas comerciales (por ejemplo, Black Friday), miles de usuarios intentan acceder y comprar simultáneamente, lo que genera problemas de concurrencia en los sistemas. Uno de los riesgos más críticos es la venta duplicada de códigos, causada por condiciones de carrera cuando múltiples transacciones acceden al mismo recurso en paralelo. Este tipo de fallos no solo impacta la experiencia del usuario, sino que también genera pérdidas económicas y problemas de confianza en la plataforma.

En este contexto, surge la oportunidad de innovar mediante el diseño de un sistema distribuido capaz de manejar grandes volúmenes de transacciones concurrentes, garantizando la consistencia de los datos y evitando errores críticos como la doble venta. La implementación de estrategias como particionamiento de datos por región (sharding), mecanismos de control de concurrencia y arquitecturas escalables permite no solo resolver estos problemas, sino también mejorar el rendimiento y la disponibilidad del sistema.

De esta forma, el desarrollo de una plataforma de venta de códigos digitales no solo responde a una necesidad del mercado, sino que representa una oportunidad para aplicar soluciones tecnológicas avanzadas en sistemas distribuidos, abordando desafíos reales de escalabilidad, consistencia y disponibilidad.

II. PROBLEMA DETECTADO

El principal problema identificado en la venta de códigos digitales de videojuegos radica en la alta concurrencia de usuarios accediendo simultáneamente al sistema, especialmente durante eventos de alta demanda. En estos escenarios, múltiples solicitudes de compra pueden intentar acceder y modificar el mismo dato (el stock de códigos) al mismo tiempo, generando lo que se conoce como una condición de carrera. Esto puede provocar que un mismo código sea vendido más de una vez, afectando directamente la experiencia del usuario y la credibilidad de la plataforma. Además, la presencia de restricciones regionales en los códigos agrega una capa adicional de complejidad, ya que el sistema debe garantizar que cada usuario acceda únicamente a productos válidos para su ubicación.

III. SOLUCIÓN PROPUESTA

Como solución, se propone el desarrollo de una plataforma web distribuida diseñada para manejar alta concurrencia de manera eficiente y segura. Esta solución contempla el uso de sharding por región geográfica para distribuir la carga del sistema, junto con mecanismos de consistencia fuerte y control de concurrencia que eviten la duplicidad en la venta de códigos. La plataforma permitirá gestionar el inventario de forma segura mediante bloqueos y reservas temporales durante el proceso de compra, asegurando que cada código sea asignado a un único usuario. Además, al tratarse de una aplicación web, se facilita el acceso global, la escalabilidad y la integración con servicios externos como pasarelas de pago, consolidando una solución robusta y alineada con las necesidades del mercado actual.

IV. DISEÑO

Para el desarrollo del sistema que proporcionara solución al problema se consideraron las siguientes estructuras dentro de este.

A. Diagrama de Arquitectura

Como se puede observar en la Fig. 1, se muestra como se conectan los distintos componentes dentro del sistema.

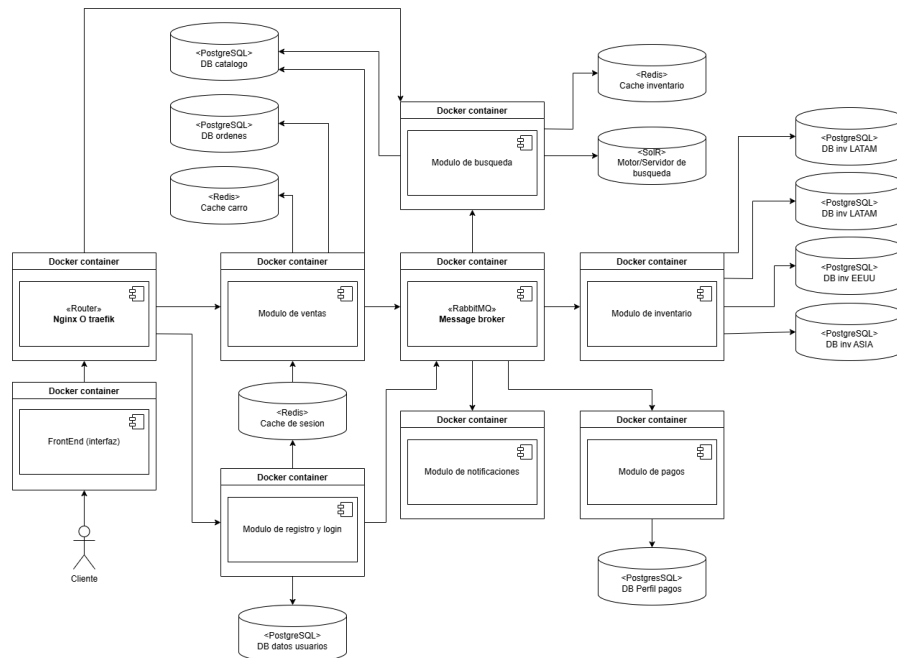


Fig. 1: Diagrama de arquitectura del sistema

B. Componentes de Software

A continuación una explicación breve de cada componente presente dentro del diagrama presentado en la Fig. 1.

- **Frontend: React Versión 19.2.x**
Interfaz gráfica donde el usuario se comunica con el sistema para la búsqueda y compra de claves digitales.
- **Balanceador de Carga: Nginx Versión 1.25.x**
Un *reverse proxy* para tomar las solicitudes del frontend y redirigirlas al módulo apropiado.
- **Message Broker: RabbitMQ Versión 3.13.x**
Gestiona una cola de eventos que pueden ser publicados y leídos por varios módulos para manejar comunicación con una arquitectura EDA (*event-driven architecture*), evitando la comunicación directa y compleja entre todos los módulos.
- **Motor/Servidor de Búsqueda: Apache SolR Versión 9.5**
Utiliza SolR como motor de búsqueda para grandes cantidades de datos, manteniendo un índice invertido.
- **Cache de Sesión: Redis Versión 7.2**
Base de datos en memoria utilizada para almacenar datos frecuentemente solicitados, agilizar procesos de inventario y carrito, además de gestionar sesiones activas.
- **Base de Datos: PostgreSQL Versión 16.x**
Almacena información en disco permitiendo su manipulación eficiente cuando sea necesaria.

En el modelo NoSQL esta principalmente lo que se guarda en memoria, siendo este la sesión actual dentro de la pagina y el carrito del usuario. Ambos estan guardados como JSON para su fácil manipulación de esto dentro del sistema. Estos se guardan con una Key y un valor asignado por el navegador

Se utiliza Redis como base de datos en memoria para gestionar sesiones y carritos de compra mediante estructuras clave-valor con datos serializados en formato JSON.

1) **Sesiones:** Permite identificar al usuario autenticado en el sistema. Cada sesión se almacena con un tiempo de expiración (TTL).

Clave:

session:<token>

Valor (JSON):

```
{
  "usuario_id": "9988-7766",
  "usuario": "SquidKid_Chile",
  "region": "LATAM",
  "correo": "correo.ejemplo@gmail.com",
  "moneda": "CLP",
  "ultimo_acceso": "2026-03-31T20:40:00Z"
}
```

Consideraciones:

- TTL asociado a la clave para expiración automática
- El token actúa como identificador único de sesión

2) **Carrito:** Almacena los productos seleccionados por el usuario antes de realizar la compra.

Clave:

cart:<usuario_id>

Valor (JSON):

```
{
  "items": [
    {
      "juego_id": "splatoon-3-ext",
      "titulo": "Splatoon 3: Expansion Pass",
      "precio": 19990,
      "cantidad": 1
    }
  ],
  "total_estimado": 19990,
  "region_compra": "LATAM"
}
```

Consideraciones:

- Asociado directamente al usuario autenticado
- Permite acceso rápido y modificaciones frecuentes

F. Preguntas para el cliente

Las preguntas que se plantearon para el posible cliente con la respuesta esperada son las siguientes para poder definir así los requisitos del sistema.

1) *¿Cuál es el volumen esperado de usuarios concurrentes en eventos pico?*

Respuesta: Se espera un volumen de hasta 50.000 usuarios concurrentes en eventos pico como Black Friday.

2) *¿Qué plataforma de pagos se utilizará y cómo se confirmarán las transacciones?*

Respuesta: Se utilizará la plataforma de pagos Stripe. Las transacciones serán confirmadas de forma asíncrona mediante webhooks, y solo tras esta confirmación se entregará el código al usuario

3) *¿El sistema debe garantizar consistencia fuerte (no vender nunca un código duplicado) o se permite consistencia eventual?*

Respuesta: Se requiere consistencia fuerte para evitar bajo cualquier circunstancia la venta duplicada de códigos.

- 4) *¿Cuál es el tiempo máximo aceptable para confirmar una compra durante alta demanda?*
Respuesta: El tiempo máximo aceptable será de 5 segundos, incluso durante eventos de alta concurrencia.
- 5) *¿Qué nivel de auditoría se necesita sobre las transacciones (registro de quién compró qué código y cuándo)?*
Respuesta: Se requiere auditoría completa, registrando usuario, producto, código entregado, fecha, hora y estado de la transacción.
- 6) *¿Cuál es el comportamiento esperado si el stock llega a 0 justo durante un pico de concurrencia?*
Respuesta: El sistema debe rechazar inmediatamente nuevas compras y notificar al usuario que el producto está agotado.
- 7) *¿Qué debe ocurrir si un usuario intenta comprar un código de una región distinta a la suya?*
Respuesta: El sistema debe bloquear la compra e informar que el producto no está disponible en su región.
- 8) *¿Qué sistema operativo utilizarán los servidores?*
Respuesta: Se utilizará Linux bajo la distribución Ubuntu Server 22.04 LTS.
- 9) *¿Con cuántos servidores contará inicialmente el sistema y cuáles son sus características?*
Respuesta: Se espera contar con 4 servidores en total, uno por cada región (LATAM, EEUU, Europa y Global), cada uno con 16 vCPU, 32 GB de RAM y 1 TB SSD, pero principalmente se contara uno en la region Global.
- 10) *¿Qué mecanismos de respaldo y recuperación se utilizarán?*
Respuesta: Se realizarán backups automáticos diarios por cada base de datos regional, con almacenamiento en la nube y capacidad de restauración ante fallos críticos.

G. Herramientas

Las Herramientas o Software que se mencionaron durante la mayor parte de este informe de manera mas simplificada y mostrando su forma de instalación para cada una son las siguientes.

La mayoría de las herramientas están instaladas sobre docker, pero existen excepciones que están instaladas dentro del servidor.

1) *Trabajan sobre Docker:* Las que están instaladas sobre Docker serian:

- **RabbitMQ 3.13.x** : Se usara la imagen proporcionada de su web oficial dentro de un contenedor

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.13-management
```
- **Nginx 1.25.x** : Se desplegara según reglas dentro del dockerfile

```
docker run -d --name nginx-router -p 80:80 -v ./nginx.conf:/etc/nginx/nginx.conf nginx:
```
- **Frontend: React 18.x (Docker)** : La versión de React se define en el archivo package.json. La aplicación se construye dentro de un contenedor utilizando Node.js y posteriormente se despliega como archivos estáticos servidos mediante Nginx.

```
docker build -t react-app .
```
- **Python + Dependencias (Docker):** Se construye una imagen basada en Python que incluye las librerías necesarias para los módulos del sistema.
 Para poder crear esto mismo se necesita de un archivo requirements.txt con lo siguiente

```
Flask
pika
psycpg2-binary
redis
pysolr
aiohttp
```

Y luego crear un dockerfile con lo siguiente para instalar todo de manera correcta

```
# Crear Dockerfile
FROM python:3.10
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Para el final solamente ejecutar el dockerfile para poder crear la imagen y poder utilizarla.

Las librerías que se utilizaron para este proyecto mencionadas anteriormente tienen una justificación por usar esas y no una alternativa.

TABLE I: Librerías utilizadas en los módulos del sistema

Librería	Descripción
Flask	Micro-framework utilizado para implementar la capa de servicios REST en cada módulo (Ventas, Inventario, Usuarios).
Pika	Librería para la comunicación con RabbitMQ, permitiendo el envío y recepción de eventos entre módulos de forma asíncrona.
psycopg2-binary	Adaptador de base de datos para PostgreSQL, utilizado para gestionar transacciones de ventas e inventario.
redis	Cliente de Python para interactuar con Redis, gestionando caché de sesiones, carritos e inventario.
pysolr	Librería para realizar consultas y gestionar índices en Solr 9.5.
aiohttp	Librería para realizar peticiones HTTP asíncronas entre módulos utilizando asyncio.

2) *Trabajan sobre el servidor:* Las siguientes son las instaladas sobre el servidor, que requieren de una actualización de los paquetes antes de instalarlos.

```
sudo apt update
```

Las herramientas sobre el servidor serian:

- **PostgreSQL 16.x** : Ejecutar el comando especificado abajo, que lo instala directamente dentro del servidor. Para permitir conexiones entre direcciones ip de contenedores docker se debe configurar postgresql.conf y pg_hba.conf

```
sudo apt install postgresql-16
```

- **redis 7.2.x** : Ejecutar el comando especificado abajo, que lo instala directamente dentro del servidor. Para cumplir con modelo de seguridad se debe editar redis.conf para cambiar directiva bind y establecer contraseña de acceso

```
sudo apt install redis-server
```

- **Java OpenJDK 17–21** : Entorno necesario para ejecutar servicios como el motor de búsqueda. Se puede instalar directamente en el servidor mediante el gestor de paquetes.

```
sudo apt install openjdk-17-jdk
```

Para usar una versión más reciente (por ejemplo 21):

```
sudo apt install openjdk-21-jdk
```

- **Apache Solr 9.5.x** : Descargar el archivo binario oficial en formato .tgz, extraerlo y ejecutar el script de instalación. Requiere Java OpenJDK (17–21) previamente instalado en el servidor.

```
wget https://downloads.apache.org/solr/solr/9.5.0/solr-9.5.0.tgz
tar xzf solr-9.5.0.tgz
cd solr-9.5.0
sudo bash ./bin/install_solr_service.sh solr-9.5.0.tgz
```